Event ends in 1 day 4 hours 5 minutes.

New Pattern: `fileMatch` Inclusion Mode

In Section 2, your EOL steering file used `inclusion: always` because EOL rules apply to every conversation. But infrastructure project context (AWS accounts, naming prefixes, tag values) only matters when you're working with `.tf` files. Loading it during a Python discussion wastes context tokens. That's what `fileMatch` solves.

	Section 2 (EOL)	Section 3 (Terraform)
Inclusion mode	`always` (loads every conversation)	`fileMatch` (loads only for `.tf` files)
Content	Universal rules + report format	Project-specific context (accounts, regions, tags)

Create the Steering File

Step 1: Create the steering directory:

macOS / Linux | Windows (PowerShell)

```
mkdir -p .kiro/steering
```

Step 2: Create `.kiro/steering/project-infra.md` (Open **terminal** if not exist and run `kiro .kiro/steering/project-infra.md` in it):

```
---
inclusion: fileMatch
fileMatchPattern: "**/*.tf"
---

# Project: MyApp Infrastructure

## Environment
- AWS Region: ap-southeast-1
- Production Account: 123456789012
- Staging Account: 987654321098
- Project prefix: myapp

## Backend Configuration
- State bucket: myapp-terraform-state
- DynamoDB lock table: myapp-terraform-locks
- State key pattern: <environment>/terraform.tfstate

## Team
- Team name: platform-engineering
- Cost center: CC-1234
- Slack channel: #myapp-infra
- On-call rotation: PagerDuty "myapp-infra"

## Tag Values for This Project
- team: platform-engineering
- cost-center: CC-1234
- project: myapp

## Project-Specific Resources
- VPC module: v5.x from registry.example.com
- RDS subnet group: myapp-db-subnet
- Shared Lambda layer: arn:aws:lambda:ap-southeast-1:123456789012:layer:myapp-common
```

fileMatch vs always
Notice `inclusion: fileMatch` with `fileMatchPattern: "**/*.tf"`. This steering file only loads when you're working with Terraform files. Compare this with Section 2's `inclusion: always` which loads in every conversation. Use `fileMatch` when your context is domain-specific.

Test It

Step 3: Download sample-infra.tf - a Terraform file with several intentional issues. Save it to your workspace.

Open the file and ask Kiro to review:

```
Review sample-infra.tf and check if it follows our project standards.
```

Kiro will automatically load the steering file because the `fileMatch` pattern matches `*.tf`. With the project context loaded, Kiro should notice issues like:

- Local backend instead of the project's S3 bucket (myapp-terraform-state) with DynamoDB locking
- Resource names ("main-vpc", "app-server", "main-db") don't use the project prefix "myapp"
- Tags only have "Name", missing required tag values (team, cost-center, project)
- Not using the specified VPC module v5.x from registry.example.com
- RDS not using the project's subnet group (myapp-db-subnet)

Kiro may also flag general issues from its own knowledge (hardcoded password, open SSH, wildcard IAM), but the steering file is what makes the review project-aware.

For Kiro CLI: run `/context show` to verify the steering file loaded due to the fileMatch pattern.

Compare with Section 2
Notice how Kiro now flags project-specific issues (naming prefix, tag values, backend config, subnet group) that it wouldn't catch without the steering file. In Section 2, the EOL steering loaded in every conversation. Here, the steering only loads when you open a `.tf` file - keeping context lean for non-Terraform work.

What's Missing
At this point, Kiro knows your project context but the review output is unstructured - it depends on how Kiro interprets your request. Remember how the EOL steering file in Section 2 was "bloated" with both rules and procedures? Here, we're starting lean from the beginning - the steering file only contains project context. The structured review process will live in the skill.

Ref: Steering Documentation